

Risk Mitigation in CI/CD Pipelines for Safety-Critical Embedded Systems Using Automated Testing and Rollback Strategies

Student name: [Insert your name]

Student number: [Insert your student number]

Campus: [Insert campus name]

Module: ITDMA3-22 Research Design and Methodology

Referencing style: Harvard author-date

Approximate article word count excluding references: 3221

Abstract

Safety-critical embedded systems increasingly rely on software updates, yet ordinary CI/CD practices cannot be transferred into these domains without additional assurance controls. A defective deployment may affect hardware behaviour, timing constraints, certification evidence and operational safety. This article reviews eight peer-reviewed studies on CI/CD in cyber-physical systems, continuous deployment in embedded systems-of-systems, automated embedded testing, mutation testing, digital twin prototypes, continuous certification and graceful degradation. The analysis argues that deployment risk should be managed through progressive assurance before deployment and controlled recovery after deployment. Automated testing reduces uncertainty by detecting defects earlier and producing repeatable evidence, while rollback, fail-safe and degraded-operation strategies reduce harm when defects still escape. The article proposes a risk-aware CI/CD pipeline that combines static and unit tests, simulation or digital twin testing, Hardware-in-the-Loop validation, mutation and coverage evidence, continuous certification artefacts, staged deployment, monitoring and predefined recovery rules.

Keywords

continuous integration; continuous deployment; safety-critical embedded systems; automated testing; rollback; graceful degradation; Hardware-in-the-Loop; digital twin prototypes

1. Introduction

Continuous integration and continuous deployment are normally valued because they shorten feedback cycles, expose defects earlier and support smaller releases. In ordinary software products, these practices can improve delivery speed and responsiveness. Safety-critical embedded systems create a different problem. Their software is connected to sensors, actuators, hardware timing, physical environments and regulatory evidence. A deployment failure may therefore produce more than inconvenience or downtime; it may create unsafe system behaviour or weaken the evidence needed to justify operational trust.

The literature shows that cyber-physical and embedded systems need a more cautious CI/CD model than web-based software. Zampetti et al. (2023) found that CI/CD in cyber-physical systems depends on a balance between continuous and periodic builds, simulator use, Hardware-in-the-Loop (HIL) testing and combined hardware-software expertise. Dakkak et al. (2023) similarly show that continuous deployment in software-intensive systems-of-systems depends on field validation, orchestration, documentation and monitoring. Baron and Louis (2023) add that safety-critical avionics development requires certification evidence to be integrated into the development process rather than treated as a late-stage audit activity.

This article investigates how automated testing and rollback strategies can reduce deployment risks in safety-critical embedded systems. The objectives are to identify key deployment risks, examine automated testing techniques suitable for embedded CI/CD pipelines, and propose a risk-aware pipeline that incorporates rollback and fail-safe mechanisms. The research question is: How can automated testing and rollback strategies within CI/CD pipelines reduce deployment risks in safety-critical embedded systems?

2. Methodology

This article uses a structured literature review approach. Sources were selected using structured search terms across academic databases and publisher repositories, including Google Scholar, IEEE Xplore, ACM Digital Library, ScienceDirect and institutional repositories. Search terms included continuous integration, continuous deployment, cyber-physical systems, embedded software testing, safety-critical software, Hardware-in-the-Loop, mutation testing, digital twin prototypes, continuous certification, rollback, fail-safe behaviour and graceful degradation. The review does not claim to be a full systematic literature review; instead, it uses a focused comparison matrix to evaluate a small but relevant set of peer-reviewed sources.

The inclusion criteria were peer-reviewed status, direct relevance to CI/CD, embedded testing, safety-critical assurance or recovery behaviour, and enough methodological detail to compare research approach, data collection, sampling and analysis. Non-peer-reviewed vendor material, blog posts and general DevOps sources without embedded or safety-critical relevance were excluded. Baker and Habli (2013) was retained despite being older because it provides strong empirical evidence about mutation testing for safety-critical airborne software, which is directly relevant to test adequacy.

Each source was analysed using the following fields: article structure, purpose, methodology, research approach, framework or model, data collection, sampling, data analysis, main findings, findings that confirm prior work, findings that extend existing research, findings that contradict or qualify earlier assumptions, literature gaps and relevance to the research question. This comparison method was influenced by the logic of systematic mapping, where literature is classified to identify patterns and gaps (Garousi et al., 2018).

Eight sources were considered sufficient for this assignment because they cover the required breadth without turning the paper into a descriptive catalogue. The selected papers represent empirical practice evidence, industrial case-study evidence, systematic mapping, prototype evaluation, mutation-testing evaluation, conceptual modelling, certification-framework work and formal recovery modelling. This spread

allows the article to compare not only what the studies found, but also how different research designs shape the strength and limits of their evidence.

3. Results: Article Structure and Methodology Comparison

The selected articles follow common research-paper sections, but their structures differ because they produce different kinds of evidence. Empirical software engineering papers such as Zampetti et al. (2023) and Dakkak et al. (2023) emphasise context, data collection, findings, discussion and validity. Systematic mapping work such as Garousi et al. (2018) gives more space to search strategy, inclusion criteria and classification. Design-science and framework papers such as Du et al. (2022), Barbie and Hasselbring (2024) and Baron and Louis (2023) foreground concepts, tools, models and demonstrations. Becker, Voss and Schätz (2018) uses a formal-methods structure because its evidence comes from modelling and constraint-based analysis.

These structural differences matter for the assignment topic because no single methodology answers the research question alone. Interviews and case studies reveal real industrial barriers, but they are context-dependent. Prototype and design-science studies show feasibility, but they may not prove broad adoption. Formal modelling supports recovery logic, but it can simplify operational complexity. Systematic mapping provides breadth, but it is less specific about one pipeline architecture. A strong answer therefore needs synthesis across evidence types.

The standard research-paper sections also serve different purposes. The abstract gives a compressed overview, the introduction frames the problem, the methodology explains how evidence was produced, the results section reports patterns, the discussion interprets meaning, the conclusion answers the research question, and the reference list makes the evidence traceable. The writing style changes across these sections: abstracts are concise, methods are objective and reproducible, while discussions are more evaluative and interpretive.

The methodology comparison also indicates how each paper contributes to the three assignment objectives. CI/CD practice studies help identify deployment risks. Testing and mutation studies help evaluate automated testing techniques. Certification and

graceful-degradation studies help design recovery and assurance mechanisms. The value of the literature is therefore cumulative: each source answers one part of the risk-mitigation problem, and the final pipeline model connects those parts.

Table 1: Research Section Purpose, Content and Style

Section	Purpose	Typical content	Writing style	Comparison note
Abstract	Summarises the whole article	Problem, method, main synthesis and recommendation	Concise and objective	All papers use it to compress contribution.
Introduction	Frames the problem and aim	Context, gap, objectives and research question	Persuasive but evidence-based	Empirical papers stress industrial context; formal papers stress modelling problem.
Literature / Background	Positions the study in prior work	Definitions, related work and theoretical or technical concepts	Analytical and source-based	Review papers make this broader; design papers make it tool/model focused.
Methodology	Explains how evidence was produced	Research design, data collection, sampling and analysis	Objective and reproducible	Systematic mapping is most explicit; formal/design papers explain models or prototypes.
Results	Reports findings or model outputs	Themes, classifications, measurements, cases or formal outcomes	Evidence-led and organised	Empirical papers report participant/case findings; formal papers report model implications.
Discussion	Interprets meaning	Agreement, contradictions, implications, limitations and gaps	Evaluative and comparative	Strongest place to connect findings to the research question.
Conclusion / References	Closes and documents evidence	Answer to research question, future work and full sources	Clear, restrained and accurate	References make Harvard in-text claims traceable.

Table 2: Methodology and Evidence-Strength Matrix

Source	Methodology	Approach	Framework /	Data / sampling	Contribution and
--------	-------------	----------	-------------	-----------------	------------------

			model		limitation
Zampetti et al. (2023)	Interviews and survey validation	Interpretivist / pragmatic	CPS CI/CD practice barriers	Interviews, card sorting, member checking and survey	Shows CPS-specific deployment and HIL/simulator risks; context-dependent.
Dakkak et al. (2023)	Industrial case study	Pragmatic	Continuous deployment in systems-of-systems	Case evidence from Ericsson 3G RAN	Shows field validation and orchestration needs; telecom-specific.
Du et al. (2022)	Engineering prototype	Design science	Automated multitest CI platform	Automotive platform demonstration	Shows feasibility of virtual/FPGA CI testing; not broad industry proof.
Baker and Habli (2013)	Empirical evaluation	Positivist	Mutation testing and test adequacy	Safety-critical C and Ada software	Strengthens assurance argument; older and not CI/CD-specific.
Garousi et al. (2018)	Systematic mapping	Positivist synthesis	Embedded testing taxonomy	312 selected technical papers	Justifies layered testing; broad rather than pipeline-specific.
Barbie and Hasselbring (2024)	Conceptual modelling and applications	Design science	Digital twin prototypes	Field/lab application evidence	Supports virtual integration tests; fidelity/certification remain open.
Baron and Louis (2023)	Framework and tooling proposal	Design science / pragmatic	Continuous certification	Industrial avionics case	Supports continuous artefacts; avionics-specific.
Becker, Voss and Schätz (2018)	Formal modelling	Formal analytical	Graceful degradation and redundancy	Constructed automotive scenario	Supports fail-safe recovery logic; not direct rollback evidence.

4. Thematic Literature Review

The first theme is deployment risk in cyber-physical and embedded contexts. Zampetti et al. (2023) show that CPS CI/CD must manage simulator limitations, HIL

availability, deployment complexity and the need for cross-disciplinary expertise. Dakkak et al. (2023) extend this into systems-of-systems by showing that continuous deployment changes field validation into an ongoing activity. Together, these studies show that deployment risk is not limited to whether a build passes. It also includes whether validation environments are realistic, whether monitoring is ready, and whether interacting systems can absorb change safely.

The second theme is layered automated testing. Garousi et al. (2018) show that embedded software testing is broad and fragmented, which supports using multiple testing techniques rather than relying on one automated stage. Du et al. (2022) demonstrate this principle through a multitest CI platform for automotive systems using virtual and FPGA-supported verification. Barbie and Hasselbring (2024) add that digital twin prototypes can support repeatable virtual integration testing when physical hardware is scarce. The shared pattern is that faster virtual tests should be complemented by higher-fidelity hardware-aware tests.

The third theme is test adequacy and assurance evidence. Baker and Habli (2013) found that mutation testing can reveal weaknesses in safety-critical airborne software even when traditional structural coverage has already been achieved. This finding is important because many CI pipelines treat a passing test suite as sufficient. In safety-critical embedded systems, the quality of the tests matters as much as the existence of tests. Baron and Louis (2023) extend the assurance argument by proposing continuous certification for safety-critical avionics, where traceability and certification artefacts are produced throughout development instead of being assembled after implementation.

The fourth theme is recovery after faults escape testing. Rollback is useful, but it is not a complete safety strategy. Becker, Voss and Schätz (2018) show that fault-tolerant automotive systems may require fail-silent, fail-safe, fail-operational or degraded modes. This qualifies common CI/CD thinking because reverting to an older version may not be the safest response if the system is already in a hazardous physical state or if hardware configuration has changed. Recovery must therefore be designed as a decision process that uses monitoring evidence, operational context and predefined safety rules.

Across the literature, a coherent pattern emerges: automated testing reduces risk before deployment, assurance artefacts make risk evidence auditable, and rollback or fail-safe mechanisms reduce harm after deployment. The contribution of this article is to connect these ideas into one risk-aware CI/CD pipeline for safety-critical embedded systems.

5. Findings, Gaps and Discussion

The findings that confirm prior research are mainly about the importance of early and repeated verification. Zampetti et al. (2023), Garousi et al. (2018) and Du et al. (2022) all support the idea that automated testing improves feedback and helps detect defects earlier. However, they also show that embedded systems require hardware-aware testing. This confirms the general value of CI/CD while qualifying it for safety-critical contexts.

The findings that extend existing research move beyond simple build automation. Dakkak et al. (2023) extend CI/CD thinking by showing that continuous deployment in systems-of-systems requires field validation and orchestration. Baron and Louis (2023) extend Agile and CI practice into certification by proposing continuous certification evidence. Barbie and Hasselbring (2024) extend simulation-based testing by formalising digital twin prototypes, while Becker, Voss and Schätz (2018) extend rollback thinking into graceful degradation and redundancy-aware recovery.

The main contradiction is not that the papers disagree about the value of CI/CD, but that they challenge the assumption that mainstream CI/CD can be transferred directly into safety-critical embedded systems. In web systems, rollback may be treated as a normal operational safeguard. In embedded safety-critical systems, rollback is only one possible response. Depending on the fault, a safer response may be degraded operation, component isolation, fail-safe shutdown or fail-operational redundancy. This contradiction exists because the consequence model is different: embedded systems interact with physical environments, and safety depends on system state as well as software version.

The reviewed papers also share an important area of agreement: pure web-style CI/CD is insufficient for safety-critical embedded systems. The articles differ in domain and method, but they agree that embedded deployment requires extra attention to hardware realism, validation evidence and system-level behaviour. This agreement directly influences the article's argument because it shifts CI/CD from a speed-focused delivery practice to a controlled assurance process.

Several literature gaps are visible when the assignment appendix categories are applied. The methodological gap is that many studies focus on a single technique or domain rather than evaluating complete end-to-end pipelines. The theoretical or conceptual gap is that CI/CD, certification and graceful degradation are rarely integrated into one shared model. The contextual gap is that evidence comes mainly from cyber-physical systems, telecommunications, automotive and avionics, so transfer to medical, industrial or other regulated domains requires caution. The data or technology gap is that digital twin prototypes and HIL environments are promising, but their fidelity and certification acceptance remain open questions. The measurement or evaluation gap is that the literature lacks a standard metric for deployment readiness that combines test adequacy, hardware realism, certification evidence, monitoring and recovery readiness. The temporal gap is visible in the reliance on Baker and Habli's older but still relevant mutation-testing evidence, which needs updating for modern high-velocity CI/CD tooling. The outcome or impact gap is that many studies focus on technical feasibility without long-term evidence of field safety outcomes. The synthesis gap is the most important: CI/CD, testing, certification and fault-tolerance research exist, but they are rarely integrated into one risk-aware deployment model.

Conceptually, the findings refine ordinary CI/CD from a delivery automation framework into a sociotechnical assurance process. The pipeline is not only a build mechanism; it is also a way of coordinating people, tools, evidence, hardware environments and recovery decisions. This theoretical implication is visible across the interview evidence in Zampetti et al. (2023), the certification framework in Baron and Louis (2023), and the formal recovery logic in Becker, Voss and Schätz (2018).

The practical implication is that risk mitigation should be evaluated before, during and after deployment. Before deployment, the question is whether test evidence is strong enough and realistic enough for the change being released. During deployment, the question is whether staged rollout and monitoring can detect abnormal behaviour early. After deployment, the question is whether the system can move to an appropriate recovery state quickly enough to prevent harm. This three-part evaluation makes the discussion more specific than a general recommendation to add more testing.

These findings influence the proposed research by shifting the focus from speed to progressive assurance. A safety-critical embedded pipeline should not ask only whether code can be deployed automatically. It should ask what evidence is required before deployment, what monitoring is needed during rollout, and what recovery decision should occur if the deployment behaves unexpectedly. This directly addresses the research question because automated testing reduces uncertainty before release, while rollback and fail-safe mechanisms limit harm after release.

Table 3: Findings and Gap Synthesis

Category	Synthesis	Evidence	Influence on research
Confirm	Layered automated testing improves feedback but must account for embedded constraints.	Zampetti et al. (2023); Garousi et al. (2018); Du et al. (2022)	Automated testing is useful only when paired with hardware-aware validation.
Extend	Continuous deployment expands into field validation, certification evidence and digital twin testing.	Dakkak et al. (2023); Baron and Louis (2023); Barbie and Hasselbring (2024)	The pipeline must produce both test results and assurance artefacts.
Contradict / qualify	Rollback is not always the safest response in physical systems.	Becker, Voss and Schätz (2018)	Recovery should include fail-safe, fail-operational and degraded modes.
Agreement	Pure web-style CI/CD is insufficient for safety-critical embedded systems.	Across the reviewed sources	CI/CD must become a controlled assurance and recovery process.
Gap	Few studies integrate CI/CD, HIL/digital twins, mutation testing, certification and recovery in one	Across reviewed literature	This article's contribution is a synthesis model for risk-aware

	model.		CI/CD.
--	--------	--	--------

6. Proposed Risk-Aware CI/CD Pipeline

A risk-aware CI/CD pipeline should begin with prevention gates. The first gate should include static analysis, build checks, unit tests and requirements traceability checks. These controls are fast and should run frequently. The second gate should include simulation and digital twin prototype tests because they provide repeatable integration feedback before scarce physical hardware is used (Barbie and Hasselbring, 2024). The third gate should include HIL or FPGA-supported validation where timing, physical interaction or hardware behaviour creates risk (Zampetti et al., 2023; Du et al., 2022).

The next layer should create assurance evidence. Mutation testing, structural coverage review, configuration evidence and certification artefact generation should be part of the pipeline rather than late documentation work. Baker and Habli (2013) show why test adequacy matters, while Baron and Louis (2023) show why certification evidence should be continuous. The deployment layer should then use staged rollout, monitoring and field validation, especially in systems-of-systems where changes may affect interactions between components (Dakkak et al., 2023).

The final layer should define controlled recovery. Rollback is suitable when monitoring detects a software regression, the previous version is known to be safe, and the hardware or configuration state remains compatible. Feature disablement is more suitable when a non-critical function fails but the core safety function remains stable. Component isolation or degraded mode is appropriate when continued operation is possible only with reduced capability. Fail-safe shutdown is needed when continued operation would be unsafe, while fail-operational redundancy is needed when stopping would create a greater hazard and redundant capacity exists. This decision logic is based on the graceful-degradation reasoning of Becker, Voss and Schätz (2018), but it is proposed here as a CI/CD recovery model that still requires empirical validation.

Monitoring is the link between deployment and recovery. For a safety-critical embedded release, monitoring should include software health signals, hardware interaction signals, timing deviations, failed safety checks, configuration drift and field-validation outcomes. These signals should not merely create alerts for developers; they should trigger predefined decisions about rollout pause, rollback, feature disablement or safe-state transition. Without this connection, monitoring becomes passive observation rather than risk mitigation.

In this model, automated testing and rollback work together rather than separately. Testing reduces the chance of failure before deployment, certification artefacts make assurance evidence auditable, staged monitoring limits exposure during release, and recovery rules reduce the consequences of failure after deployment. The model is therefore best understood as a proposed synthesis of the reviewed literature, not as a fully validated industrial standard.

Table 4: Proposed Pipeline Gates

Stage	Gate	Main controls	Risk function
1	Commit and build	Static analysis, build checks, unit tests and traceability checks	Fast prevention
2	Simulation / digital twin	Repeatable virtual integration tests	Early integration confidence
3	HIL / FPGA validation	Hardware-aware timing and physical-interaction tests	Environmental realism
4	Assurance evidence	Mutation, coverage, configuration and certification artefacts	Auditable safety evidence
5	Staged deployment	Limited rollout, monitoring and field validation	Controlled exposure
6	Recovery decision	Rollback, feature disablement, isolation, degraded mode or fail-safe behaviour	Post-deployment harm reduction

7. Journal or Conference Venue

A suitable venue for this article would be the International Conference on Software Testing, Verification and Validation (ICST). ICST is appropriate because its scope includes software testing, verification, validation, empirical studies, tools, embedded software and technology transfer (ICST Steering Committee, 2023). The article's focus on automated testing, HIL, mutation testing, digital twin test environments and deployment risk in embedded systems fits this venue better than a broad software engineering venue. ICST should be treated as the venue source only, not as one of the eight peer-reviewed research papers.

8. Conclusion

Automated testing and rollback strategies can reduce deployment risks in safety-critical embedded systems when they are implemented as part of a broader risk-aware CI/CD pipeline. The literature shows that embedded CI/CD requires more than fast builds and frequent releases. It requires layered automated testing, hardware-aware validation, test adequacy evidence, continuous certification artefacts, staged deployment, monitoring and predefined recovery behaviour. Rollback is valuable, but it should be treated as one recovery option rather than a universal solution. In safety-critical systems, degraded operation, fail-safe shutdown or fail-operational redundancy may be safer than simple version reversal. The main contribution of this article is therefore a synthesis model: progressive assurance before deployment plus controlled recovery after deployment.

References

- Baker, R. and Habli, I. (2013) 'An empirical evaluation of mutation testing for improving the test quality of safety-critical software', *IEEE Transactions on Software Engineering*, 39(6), pp. 787-805. Available at: <https://doi.org/10.1109/TSE.2012.56>.
- Barbie, A. and Hasselbring, W. (2024) 'From digital twins to digital twin prototypes: Concepts, formalization, and applications', *IEEE Access*, 12, pp. 75337-75365. Available at: <https://doi.org/10.1109/ACCESS.2024.3406510>.

- Baron, C. and Louis, V. (2023) 'Framework and tooling proposals for Agile certification of safety-critical embedded software in avionic systems', *Computers in Industry*, 148, Article 103887. Available at: <https://doi.org/10.1016/j.compind.2023.103887>.
- Becker, K., Voss, S. and Schätz, B. (2018) 'Formal analysis of feature degradation in fault-tolerant automotive systems', *Science of Computer Programming*, 154, pp. 89-133. Available at: <https://doi.org/10.1016/j.scico.2017.10.007>.
- Dakkak, A., Bosch, J., Holmström Olsson, H. and Mattos, D.I. (2023) 'Continuous deployment in software-intensive system-of-systems', *Information and Software Technology*, 159, Article 107200. Available at: <https://doi.org/10.1016/j.infsof.2023.107200>.
- Du, B., Azimi, S., Moramarco, A., Sabena, D., Parisi, F. and Sterpone, L. (2022) 'An automated continuous integration multitest platform for automotive systems', *IEEE Systems Journal*, 16(2), pp. 2495-2506. Available at: <https://doi.org/10.1109/JSYST.2021.3069548>.
- Garousi, V., Felderer, M., Karapıçak, Ç.M. and Yılmaz, U. (2018) 'Testing embedded software: A survey of the literature', *Information and Software Technology*, 104, pp. 14-45. Available at: <https://doi.org/10.1016/j.infsof.2018.06.016>.
- ICST Steering Committee (2023) 'The International Conference on Software Testing, Verification and Validation - ICST'. Available at: <https://icstconference.github.io/> (Accessed: 4 June 2026).
- Zampetti, F., Tamburri, D.A., Panichella, S., Panichella, A., Canfora, G. and Di Penta, M. (2023) 'Continuous integration and delivery practices for cyber-physical systems: An interview-based study', *ACM Transactions on Software Engineering and Methodology*, 32(3), pp. 1-44. Available at: <https://doi.org/10.1145/3571854>.